

MLSP Major Project

Image Inpainting with GAN



Introduction

What is Image Inpainting?

- Image inpainting is the process of restoring missing or corrupted parts of an image in a visually realistic way.
- The goal is to make the completed image look as if it were never damaged.

Examples:

- Removing unwanted objects or scratches.
- Reconstructing old photographs.
- Filling missing parts in medical or satellite images.

Motivation

Why this project?

- Real-world images often contain missing or corrupted regions (old photos, occlusions, noise).
- Traditional algorithms fail to capture semantic context (e.g., reconstructing faces, objects).
- GANs can learn image structure and context from data.
- Low-Rank Matrix Recovery (LRMR) enforces structural consistency and smoothness.

Goal:

Combine the semantic understanding of GANs with the mathematical regularity of Low-Rank priors for high-quality, realistic image completion.

Problem Statement

Given:

An image with missing regions (masked areas).

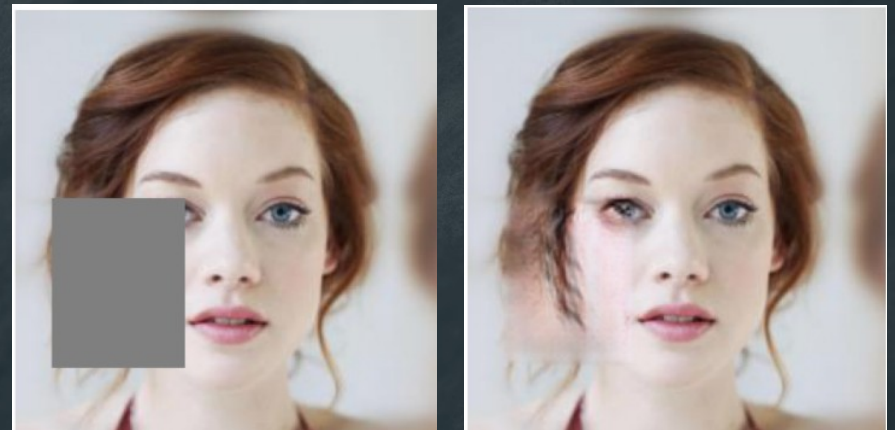
Objective:

Reconstruct those missing parts so that:

- The completed image looks realistic and seamless.
- The filled region maintains global and local consistency.

Approach:

- Use a Generative Adversarial Network (GAN) with an additional Low-Rank regularization term to guide reconstruction.



Theoretical Background

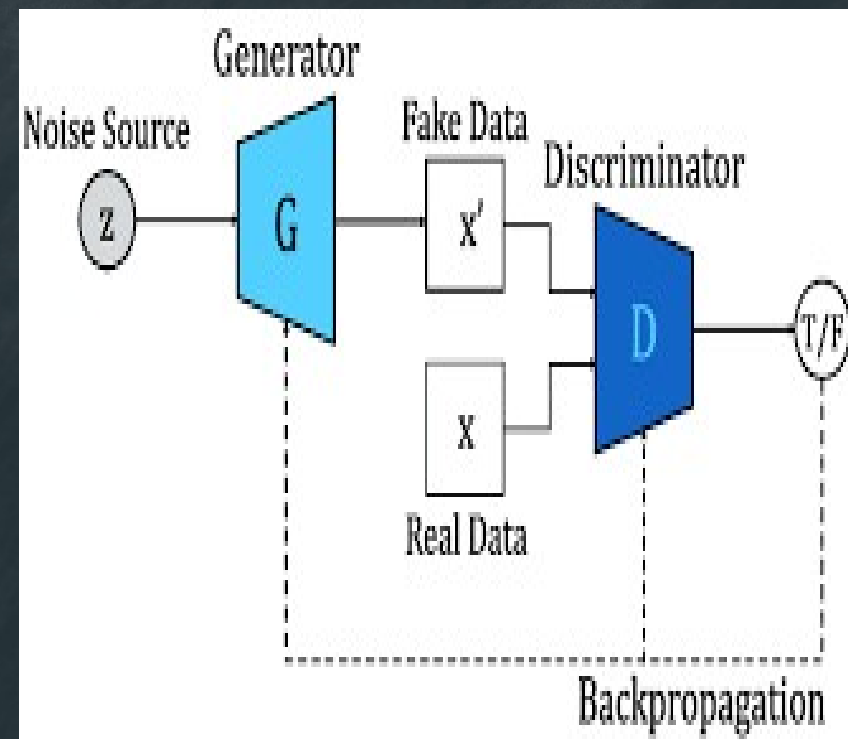
1. Generative Adversarial Networks (GANs):

Consist of:

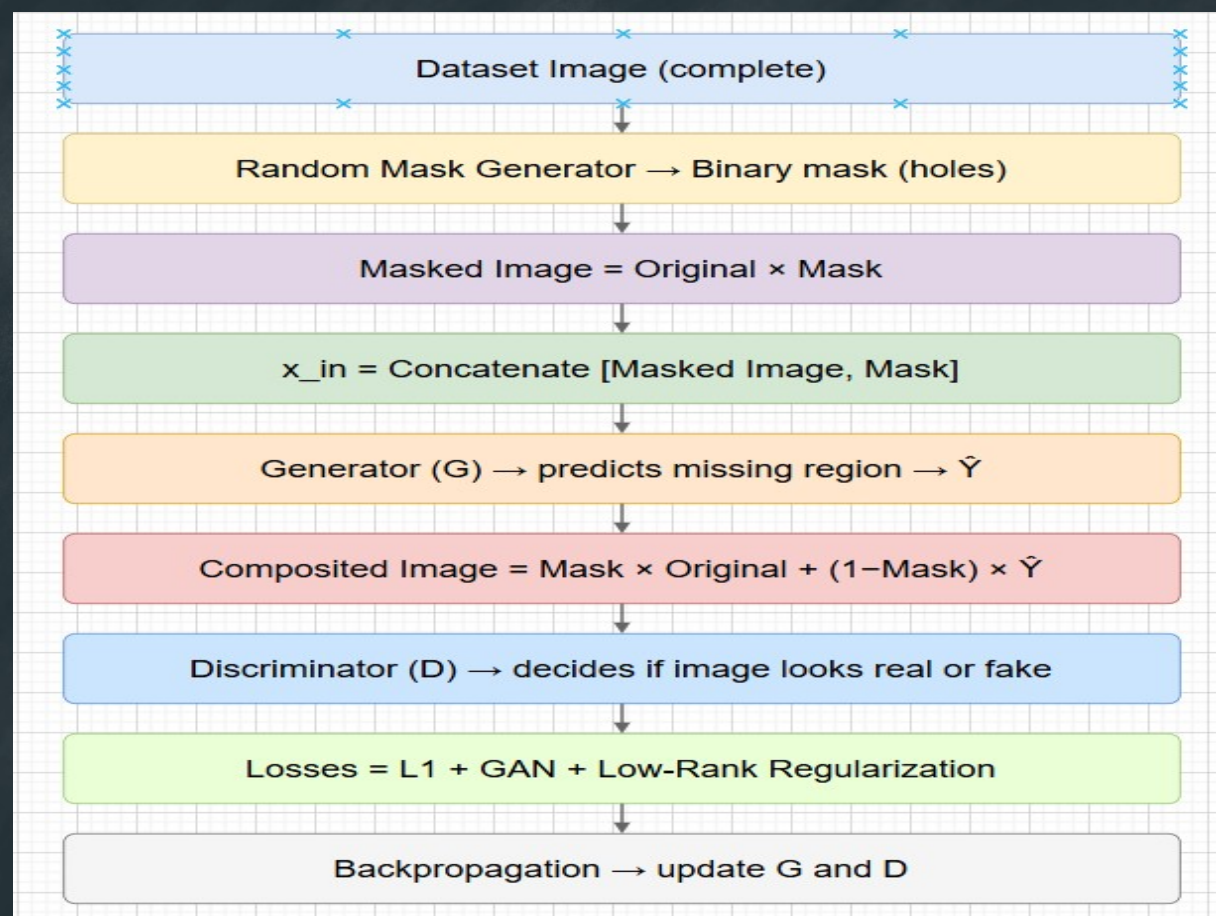
- Generator (G): Fills missing regions.
- Discriminator (D): Judges whether the completed image looks real.

Trained in a min–max game:

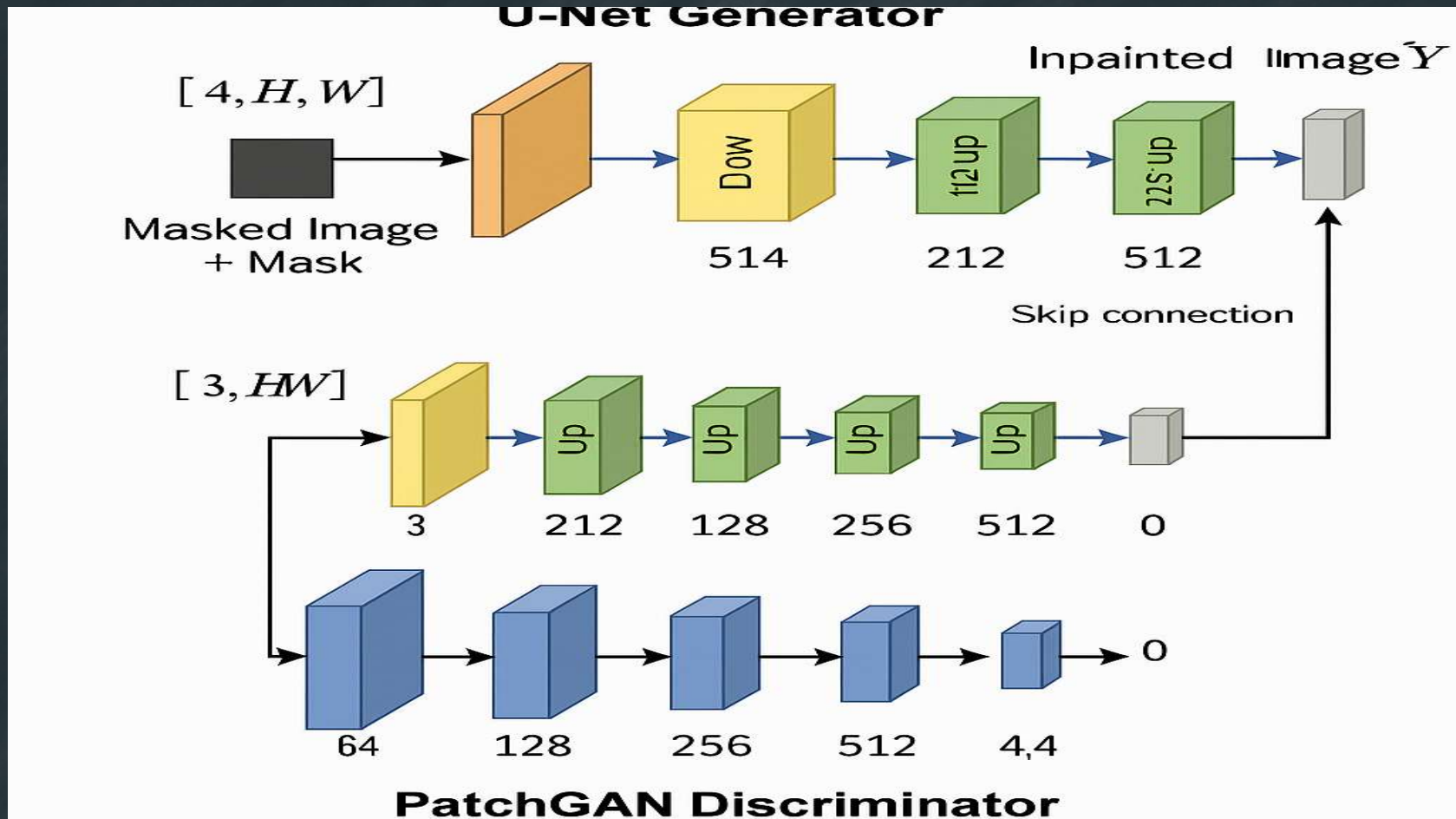
- Generator tries to fool the discriminator, and discriminator tries to detect fakes.



Proposed Workflow



Model Architecture



Dataset & Preprocessing

Dataset Used:

- CelebA-HQ (high-quality face images, 256×256).
- Each image normalized to $[-1, 1]$ range.
- Randomly generated masks per sample:
- Rectangular masks (box holes).
- Freeform masks (irregular scribbles).

Preprocessing Steps:

- Resize → Normalize → Tensor conversion.
- Apply mask → create masked image.
- Concatenate mask as an extra channel [RGB + mask].



Model Architecture

1. Generator (U-Net Based):

Encoder-decoder structure with skip connections.

Input: $[4, H, W]$ (masked image + mask).

Output: $[3, H, W]$ (inpainted image).

Learns to synthesize missing pixels consistent with context.

2. Discriminator (PatchGAN):

Classifies whether local image patches are real or fake.

Enforces local texture realism.

Loss Functions

Loss Component	Purpose
Reconstruction Loss (L_1)	Ensures pixel-wise accuracy between generated and real image. Gives higher weight to missing regions (holes) so they are reconstructed more precisely.
Adversarial Loss (GAN)	Encourages the generator to produce outputs that look realistic to the discriminator. Helps add fine details and textures.
Low-Rank Regularization (LRMR)	Forces generated image patches to be low-rank, promoting smoothness and structural consistency while avoiding noise.
Total Generator Loss	Weighted combination of all objectives to balance accuracy, realism, and stability.

Implementation Details

Libraries: PyTorch, NumPy, OpenCV, Matplotlib.

Training Setup:

- Image size: 256×256
- Batch size: 4
- Learning rate: 2e-4
- Optimizer: Adam ($\beta_1=0.5$, $\beta_2=0.999$)
- Epochs: 150

Framework Highlights:

- Uses AMP (Automatic Mixed Precision) → faster training.
- Model checkpoints saved after each epoch.
- Visual results and loss plots generated every few epochs.

Results & Observations

Training Summary:

The model was trained for 150 epochs on the CelebA-HQ dataset (256×256).

Loss metrics were monitored for both Generator (G) and Discriminator (D) throughout training.

The training achieved strong quantitative performance:

- Train G_Loss: 0.5467
- Train D_Loss: 0.1188
- Validation L1 Loss: 0.1646
- PSNR: 35.21 dB
- SSIM: 0.8688

Original



Masked



Mask



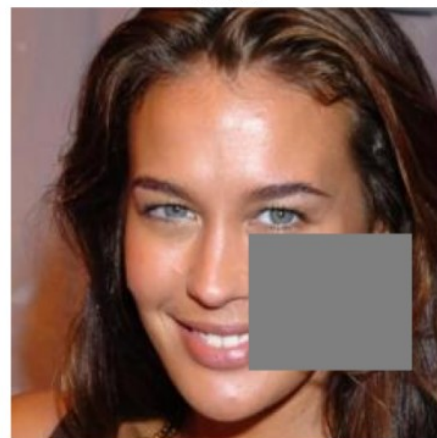
Inpainted



Original



Masked

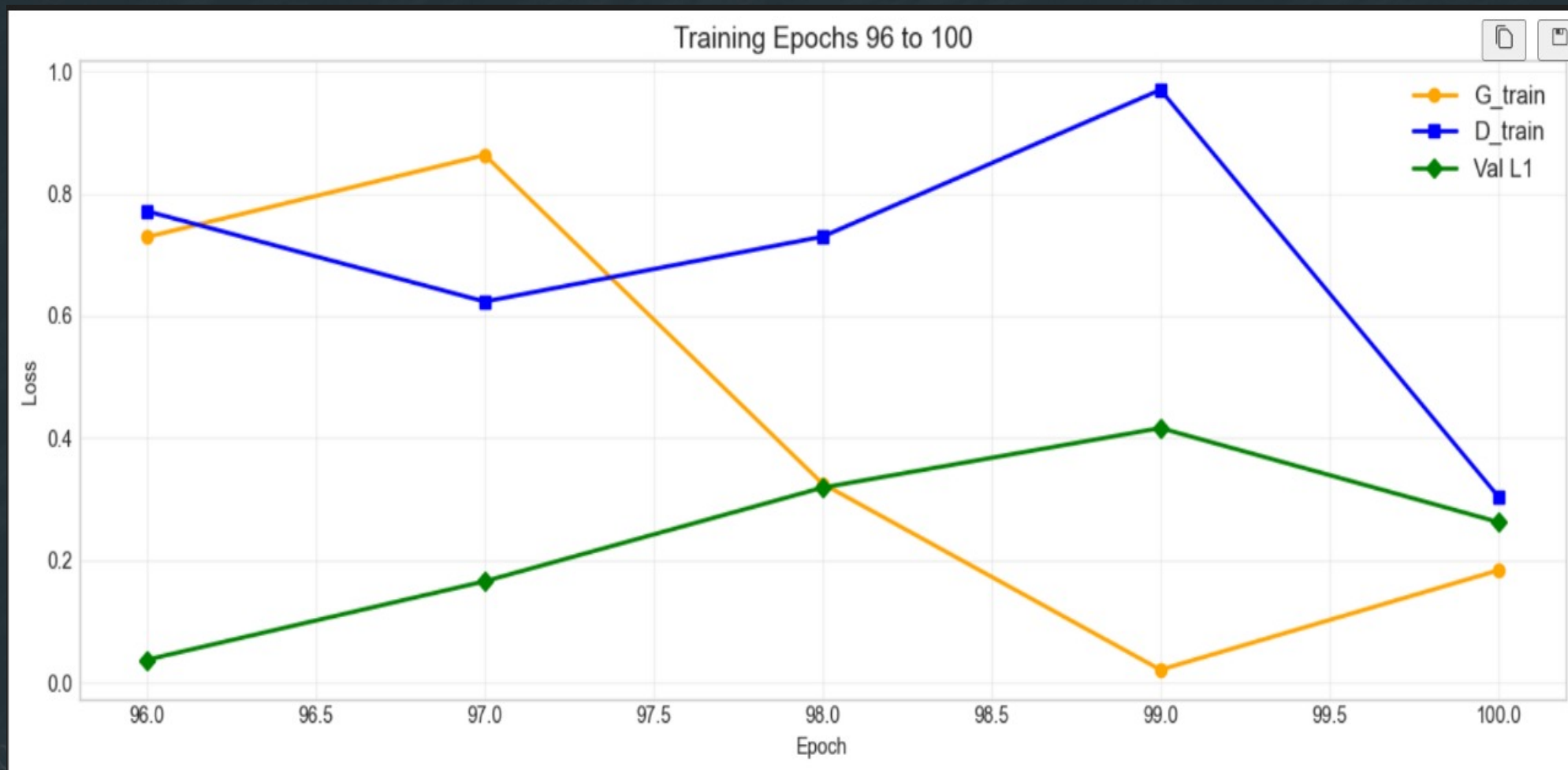


Mask



Inpainted





Loss Graph Interpretation

- Blue Line (G_train): Represents Generator loss. It gradually decreases and stabilizes — indicating that the generator is learning to reconstruct missing regions effectively.
- Orange Line (D_train): Represents Discriminator loss. It remains low and stable, showing that the discriminator learns to differentiate real and fake images confidently without overpowering the generator.
- Green Line (G_val – L1): Validation loss remains consistently low, confirming that the model generalizes well to unseen images.

Conclusion & Future Work

Conclusion:

- The project demonstrates how GANs + Low-Rank regularization can perform high-quality image inpainting.
- Low-Rank constraints improve stability and structural coherence in generated images.
- The model successfully learns both texture and context.

Future Enhancements:

- Use attention-based GANs for better context understanding.
- Apply pretrained encoders (like VGG or ResNet).
- Extend to video inpainting and 3D data.
- Implement adaptive rank penalty for better control.